

機器人學 Project1

313512009 施丞蔚 william0726.tai@gmail.com

一、介面說明（開發平台、如何執行...）(10%)

- 執行環境：Windows 11
- 開發工具：Python 3.12.6 64-bit
- 編譯器：VS Code
- 如何執行：直接透過 VS Code 內建的編譯器執行，或是在終端機使用 `python Project1_1.py` 與 `Project1_2.py`。

二、程式架構說明（程式運行流程、核心程式碼說明...）(20%)

順向運動學(Project1_1.py):

- 程式運行流程：
 1. 匯入模組：首先匯入 `numpy` 與數學函數模組，這些模組主要用於矩陣計算與三角運算。
 2. 定義 DH 參數表：使用 Denavit-Hartenberg (DH) 參數定義六個關節的結構。
 3. 定義正向運動學函數：計算六軸機械臂末端執行器的齊次變換矩陣，從而得到其位置與姿態。
 4. 計算與顯示結果：對於給定的兩組關節角度，使用正向運動學函數來計算並顯示結果。
- 核心程式碼說明：
 1. DH 參數表 `DH_TABLE`：儲存每個關節的 DH 參數，包括距離 `d`、連接長度 `a`、旋轉角度 `alpha` 和 `theta`，這些參數用於描述各個連桿間的相對位置。
 2. `forward_kinematics` 函數：
 - I. 透過六個關節的參數來計算每個連桿的齊次變換矩陣 A_i ，並逐步相乘得到最終的齊次變換矩陣 T 。
 - II. 提取旋轉矩陣的三個列向量 n 、 o 、 a ，以及位置向量 p ，這些向量描述末端執行器的姿態和位置。
 - III. 使用 Z-Y-Z 歐拉角方式計算末端的旋轉角度（ ϕ 、 θ 、 ψ ）。
 3. 計算結果輸出：顯示旋轉矩陣和末端執行器的位置、姿態。

4. 結果:

```
PS C:\Users\willi\Desktop> & C:/Users/willl/AppData/Local/Programs/Python/Python312/python.exe
c:/Users/willl/Desktop/機器人學/Project1/Project1_1.py
[n, o, a, p] for input_T1:
0.7798 -0.6257 -0.0194 577.8579
-0.0582 -0.1033 0.9930 210.2800
-0.6233 -0.7732 -0.1169 -85.4896
0.0000 0.0000 0.0000 1.0000
output:
577.8579 210.2800 -85.4896 -51.1252 96.7153 91.1177

[n, o, a, p] for input_T2:
0.1704 -0.9695 -0.1760 347.3731
-0.5630 -0.2424 0.7901 414.2652
-0.8087 -0.0356 -0.5872 -191.5650
0.0000 0.0000 0.0000 1.0000
output:
347.3731 414.2652 -191.5650 -2.5193 125.9553 102.5577
```

逆向運動學(Project1_2.py):

● 程式運行流程：

1. 引入必要的模組：包括 `math` 和 `numpy`，用於數學運算和矩陣操作。
2. 設置 `numpy` 輸出格式：禁止科學記數法，以便顯示更直觀的輸出。
3. 定義函數 `print_joint_angles(cta)`：
 - I. 負責輸出計算得到的各個關節角度，並檢查是否有角度超出可操作的範圍。
 - II. 如果角度超出範圍，會顯示相關的提示。
4. 定義函數 `calculate_theta(A)`：
 - I. 接收機器人末端姿態的 4×4 齊次變換矩陣作為輸入，並計算機器人的各關節角度。
 - II. 內部包含多個子函數 `solution_1` 到 `solution_8`，代表八組不同的解法，用於求解所有可能的解。
5. 八組解的計算：
 - I. 透過各種不同的角度配置，每個子函數計算一組完整的關節角度，以涵蓋所有可能的逆向解。
6. 運行主體：
 - I. 程式最後定義了兩個變換矩陣 `A1` 和 `A2`，分別代表機器人末端在空間中的不同位置和方向。
 - II. 調用 `calculate_theta` 函數，對這兩個矩陣計算相應的各關節角度，從而輸出不同解的結果。

7. 結果：

```
PS C:\Users\willi\Desktop> & C:/Users/willi/AppData/Local/Programs/Python/Python312/python.exe
c:/Users/willi/Desktop/機器人學/Project1/Robotics_Project1_313512009/Project1_2.py
-A1關節角度-----
  60.000   -28.500   115.000  -121.500    10.000    50.000
Joint(s) 3 4 out of range

  60.000   -28.500   115.000    58.500   -10.000  -130.000
Joint(s) 3 out of range

  60.000    90.000  -115.000   -10.000    10.000    50.000

  60.000    90.000  -115.000   170.000   -10.000  -130.000
Joint(s) 4 out of range

 -120.000   149.800    27.100    38.100   170.000  -130.000
Joint(s) 2 3 out of range

 -120.000   149.800    27.100  -141.900  -170.000    50.000
Joint(s) 2 3 4 out of range

 -120.000   177.400   -27.100    64.700   170.000  -130.000
Joint(s) 2 out of range

 -120.000   177.400   -27.100  -115.300  -170.000    50.000
Joint(s) 2 4 out of range

-A2關節角度-----
  90.000   -23.800   119.000  -125.200    10.000    0.000
Joint(s) 3 4 out of range

  90.000   -23.800   119.000    54.800   -10.000  -180.000
Joint(s) 3 out of range

  90.000    99.000  -119.000   -10.000    10.000    0.000

  90.000    99.000  -119.000   170.000   -10.000  -180.000
Joint(s) 4 out of range

 -90.000   137.900    44.300    27.800   170.000   180.000
Joint(s) 2 3 out of range

 -90.000   137.900    44.300  -152.200  -170.000    -0.000
Joint(s) 2 3 4 out of range

 -90.000   176.900   -44.300    77.300   170.000   180.000
Joint(s) 2 out of range

 -90.000   176.900   -44.300  -102.700  -170.000    -0.000
Joint(s) 2 out of range
```

● 核心程式碼說明：

1. `print_joint_angles(cta)`

這個函數的主要作用是將從弧度轉換得到的角度進行格式化輸出，同時檢查各關節角度是否超出機器人機械結構的物理限制。例如，對於每個關節其活動範圍用 `limits` 變數進行定義，如下：

- `cta1` 在 $[-150, 150]$ 度之間，`cta2` 在 $[-30, 100]$ 度之間，依此類推。這些限制確保關節不會超出安全的操作範圍。

2. `calculate_theta(A)`

該函數是程式的核心，負責逆向運動學的計算。它依次計算各種不同解法的可能性，共有 8 種解法。每種解法的過程大致如下：

- 首先提取出 4×4 變換矩陣中的位置和姿態分量。
- 使用反正切和反餘弦等基本三角函數，基於末端姿態計算各關節角度。
- 在此過程中，涉及三個主要臂段長度參數 `a1`, `a2`, `a3`，分別表示各段連桿的長度。

3. 每個 `solution` 子函數邏輯有所不同，而它們共同計算步驟如下：

該函數是程式的核心，負責逆向運動學的計算。它依次計算各種不同解法的可能性，共有 8 種解法。每種解法的過程大致如下

- `cta1` 計算：根據末端的位置 (`Px`, `Py`)，利用 `atan2` 計算出第一關節的旋轉角度。
- `cta5` 計算：通過末端姿態向量與第一關節的旋轉角，使用 `acos` 函數計算出第五關節的角度。
- `cta6` 計算：根據末端姿態矩陣中的 `ox`, `oy` 等元素計算末端旋轉軸的方向角。
- `cta3` 和 `cta2` 計算：利用餘弦定理計算第三和第二關節的角度 `cta3` 通常使用 `acos` 計算，而 `cta2` 則通過逆運算得到。
- `cta4` 計算：根據之前計算得到的角度，以及末端姿態的方向，進一步使用 `atan2` 得到第四關節的角度。

4. 八組解的意義

在機器人的逆向運動學中，通常對於末端的位置和姿態，會有多組可行的解法。這些不同的解法對應著機器人手臂不同的姿態，例如肘部抬高或抬低等。因此該程式通 `solution_1` 到 `solution_8` 來遍歷所有可能的關節配置，保證計算得到所有可能的解。

三、數學運算說明、推導（請根據你的程式推導，不要偷用黑魔法）(30%)

順向運動學(Project1_1.py):

1. Denavit-Hartenberg (DH) 參數法：

- DH 參數法是一種用於描述多關節機械臂各連桿之間相對位置的標準化方法。每個關節由四個參數來描述：d (前臂長度)、a (連桿長度)、alpha (連桿扭轉角) 和 theta (關節角)。
- 齊次變換矩陣 A_i 的計算基於 DH 參數：

$$A_i = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \cos(\alpha) & \sin(\theta) \sin(\alpha) & a \cos(\theta) \\ \sin(\theta) & \cos(\theta) \cos(\alpha) & -\cos(\theta) \sin(\alpha) & a \sin(\theta) \\ 0 & \sin(\alpha) & \cos(\alpha) & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 將所有六個關節的變換矩陣相乘，得到最終的末端執行器的齊次變換矩陣 T。

2. 正向運動學推導：

- 齊次變換矩陣 T：
 - 繼續相乘每個連桿的變換矩陣，最終得到一個總變換矩陣 T，描述從基座到末端執行器之間的位姿變換。
 - T 的形式為：

$$T = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中，n、o、a 分別為旋轉矩陣的三列向量，p 為位置向量。

3. 計算 Z-Y-Z 歐拉角：

- 歐拉角的計算是根據 T 中的旋轉矩陣部分來確定末端執行器的姿態。
- 使用 Z-Y-Z 旋轉順序，對於旋轉矩陣中描述的各列向量，我們使用以下方法來計算：
 - 第一個旋轉角 (ψ)：計算繞 Z 軸的旋轉，使用 $\text{atan2}(T[1, 2], T[0, 2])$ 。
 - 第二個旋轉角 (θ)：計算繞 Y 軸的旋轉，使用 $\text{atan2}(\sqrt{T[0, 2]^2 + T[1, 2]^2}, T[2, 2])$ 。
 - 第三個旋轉角 (ϕ)：計算繞 Z 軸的旋轉，使用 $\text{atan2}(T[2, 1], -T[2, 0])$ 。
- 最後將這些角度從弧度轉換為角度。

逆向運動學(Project1_2.py):

1. 齊次變換矩陣 A 的定義：

$$A = \begin{bmatrix} nx & ox & ax & Px \\ ny & oy & ay & Py \\ nz & oz & az & Pz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- nx, ny, nz 代表 n 向量的分量，即機器人末端在空間中的 x, y, z 方向上的法向量。
- ox, oy, oz 代表 o 向量的分量，即機器人末端在空間中的 x, y, z 方向上的橫向向量。
- ax, ay, az 代表 a 向量的分量，即機器人末端在空間中的 x, y, z 方向上的姿態向量。
- Px, Py, Pz 代表機器人末端在空間中的位置。

2. 計算各個關節角度的步驟：

- **cta1 (第一關節)的推導：**

cta1 通常是基於末端位置 Px 和 Py 計算出來，代表機器人基座旋轉的角度。使用的公式為：

$$cta1 = \text{atan2}(Py, Px)$$

這個公式可以計算出機器人的基座如何旋轉來對準末端的目標位置， atan2 函數能夠確保計算結果位於正確的象限中。

- **cta5 (第五關節)的推導：**
- cta5 的推導主要基於末端的姿態向量 ax 和 ay ，以及第一關節的旋轉結果：

$$cta5 = \text{acos}(-\sin(cta1) \cdot ax + \cos(cta1) \cdot ay)$$

這個公式用於計算末端姿態在水平方向上的偏移。由於第五關節通常是控制機器人腕部的姿態，因此它與姿態向量的投影有關。

- **cta6 (第六關節)的推導：**
- cta6 代表機器人末端旋轉的方向，主要由 ox, oy, nx, ny 和 cta1 計算得出：

$$cta6 = \text{atan2}(-\sin(cta1) \cdot ox + \cos(cta1) \cdot oy, \sin(cta1) \cdot nx - \cos(cta1) \cdot ny)$$

此角度表示機器人末端的橫向旋轉，與姿態矩陣中的元素有關。

- **cta3 (第三關節)的推導：**
- cta3 通常是基於餘弦定理來計算，這裡使用了機器人臂長參數 a_1, a_2, a_3 和末端位置 P_x, P_y, P_z 。餘弦定理公式如下：

$$\cos(\text{cta3}) = \frac{Px^2 + Py^2 + Pz^2 - 2a_1(\cos(\text{cta1}) \cdot Px + \sin(\text{cta1}) \cdot Py) + a_1^2 - a_3^2 - a_2^2}{2a_2a_3}$$

cta3 可以通過 acos 來計算，這表示第三個關節的角度，它控制機器人的肘部位置。

- **cta2 (第二關節)的推導：**
- cta2 的推導是基於 cta3、 P_x, P_y, P_z 以及臂長 a_2 和 a_3 。公式為：

$$\cos(\text{cta2}) = \frac{a_3 \cos(\text{cta3})(\cos(\text{cta1})Px + \sin(\text{cta1})Py - a_1) - a_3 \sin(\text{cta3})Pz + a_2(\cos(\text{cta1})Px + \sin(\text{cta1})Py - a_1)}{(\cos(\text{cta1})Px + \sin(\text{cta1})Py - a_1)^2 + Pz^2}$$

cta2 使用 acos 計算，代表機器人肩部的角度。

- **cta4 (第四關節)的推導：**
- cta4 通常是基於 cta1、cta2、cta3 以及末端姿態向量來計算，使用 atan2 函數：

$$\text{cta4} = \text{atan2}(-\cos(\text{cta1})\sin(\text{cta2} + \text{cta3}) \cdot ax - \sin(\text{cta1})\sin(\text{cta2} + \text{cta3}) \cdot ay - \cos(\text{cta2} + \text{cta3}) \cdot az, \cos(\text{cta1})\cos(\text{cta2} + \text{cta3}) \cdot ax + \sin(\text{cta1})\cos(\text{cta2} + \text{cta3}) \cdot ay - \sin(\text{cta2} + \text{cta3}) \cdot az)$$

這個角度控制機器人手腕的旋轉方向。

四、加分題：討論兩種逆向運動學(代數法，幾何法)的優缺點 (10%)

代數法 (Algebraic Method) 的優缺點：

優點：

1. **普遍適用：**代數法可以用於多種機器人結構，無論是串聯型或並聯型，具有一定的普適性。
2. **系統化求解：**可以對整個逆向運動學問題進行系統性的方程求解，適合求得所有可能解。
3. **適合自動化求解：**適合運用計算工具（如符號計算軟件）進行自動化解決，非常數學化，且可導出顯式方程式。

缺點：

1. **計算複雜：**方程組的數學求解過程較複雜，特別是在多自由度的情況下，容易變成高階非線性方程組，求解過程可能需要大量的計算資源。
2. **不直觀：**代數法主要基於矩陣和方程組求解，不容易直觀理解，也不便於手動計算和即時調整。
3. **多解性處理困難：**機器人的逆向運動學通常會有多組解，代數法在求解

過程中難以處理和選擇最優解，特別是在物理限制和機構工作範圍存在的情況下。

幾何法 (Geometric Method) 的優缺點：

優點：

1. **直觀易理解：**幾何法基於機器人結構的幾何關係，通過視覺化方式（如三角函數和向量）來求解，因此較容易理解和手動求解。
2. **求解速度快：**因為不涉及複雜的方程組求解，通常可以較快地找到解，適合即時運算和控制應用。
3. **多解性清晰：**可以通過圖形分析來直觀地理解多解情況，容易掌握和判斷不同解的物理意義，尤其是在處理機器人的運動極限時很有幫助。

缺點：

1. **局限性大：**幾何法僅適用於特定的機器人結構，通常是對稱性較強的簡單結構（如六自由度機械臂），在結構較複雜時很難應用。
2. **難以系統化處理：**當機器人自由度較高時，幾何法不容易系統性地處理所有自由度，且在三維空間中畫圖解算也變得困難。
3. **精度限制：**幾何法依賴於圖形的可視化和三角關係，對於非常精確的求解可能存在一定的誤差。

五、參考資料

機器人學講義-楊谷洋老師